

Software Architecture and Design Specification

Project: Student Attendance Management System (SAMS)

Subject Code: UE24CS341A — Software Engineering

Version: 1.0

Authors: Piyush A Patel; Neel Chandrakar; Nookala Sai Tushar Krishna; Pranav Somashekar

Date: 03-10-2026

Status: Draft — For Submission

Project Team

USN	Name
PES2UG24CS350	Piyush A Patel
PES2UG24CS310	Neel Chandrakar
PES2UG24CS331	Nookala Sai Tushar Krishna
PES2UG24CS364	Pranav Somashekar

Revision History

Version	Date	Author	Change Summary
1.0	03-10-2026	Piyush A Patel	Initial SAD draft for SAMS, aligned to course SAD template

Approvals

Role	Name	Signature/Date
Course Coordinator		

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the Student Attendance Management System (SAMS). It describes the system's component structure, chosen architecture pattern, data design, interface design, and security architecture, translating the functional and non-functional requirements defined in the Phase-1 SRS into a concrete technical design that the team will implement.

1.2 Scope

Covers the web application's internal architecture: the Django-based presentation, application, and data layers; the module decomposition for Authentication, Master Data Management, Attendance Engine, Analytics & Reporting, and Dashboards; and the key interaction flows between them. It does not cover deployment/infrastructure architecture beyond the local/WSGI setup already defined in the SRS, since cloud deployment is explicitly out of scope for this project.

1.3 Audience

Student developers (project team), the project guide/evaluator, and any future contributor who extends or maintains the system after the initial submission.

1.4 Definitions

SAMS, SRS, SAD, UML, ORM, RBAC, CSRF, STRIDE, WBS. Full definitions are provided in the Phase-1 SRS, Section 1.4.

2. Document Overview

2.1 How to use this document

This document provides the architectural and design deliverables for SAMS, including UML use-case, component, and sequence diagrams, the chosen architecture pattern and rationale, API/endpoint design, and a security (threat-modelling) architecture. It is intended to be read alongside the Phase-1 SRS (for requirements) and the Phase-2 Project Plan (for schedule and team ownership).

2.2 Related Documents

- Software Requirements Specification (SRS) — Student Attendance Management System, Version 1.0: [GitHub — SRS-Student Attendance Management System.pdf](#)
- Jackfruit Phase-2 — Project Plan Document, Version 1.1
- Jackfruit Mini-Project Guidelines, Software Engineering (UE24CS341A), 2026

3. Architecture

3.1 Goals & Constraints

Goals: a reliable, maintainable, role-secure web application that correctly computes and reports attendance for a single institution at classroom scale.

Constraints: must be implemented in Python/Django with SQLite as the only database (per course requirements); frontend must use Django Templates rather than a separate SPA framework; development

and evaluation are confined to a single academic semester; no biometric, facial-recognition, AI/ML-based, or cloud-hosted components are in scope.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Administrator	Accurate master data, reliable institution-wide reporting, simple account management
Faculty	Fast, error-free attendance marking; correct low-attendance alerts for their sections
Student	Accurate, up-to-date, and private view of their own attendance
Course Evaluator / Instructor	Traceability from requirements to design to tests; sound architectural rationale
Development Team	Modularity, maintainability, and independent workstreams across four members

3.3 UML Diagrams

Two categories of UML diagram are provided: use-case diagrams (actor-level scope, carried over from the SRS for architectural traceability) and a component diagram (internal module structure, new to this document).

3.3.1 Use-Case Diagrams

The following two use-case diagrams, originally presented in SRS Section 7.1, are reproduced here for architectural completeness, since every subsystem in Section 3.4 exists to realise one or more of these use cases.

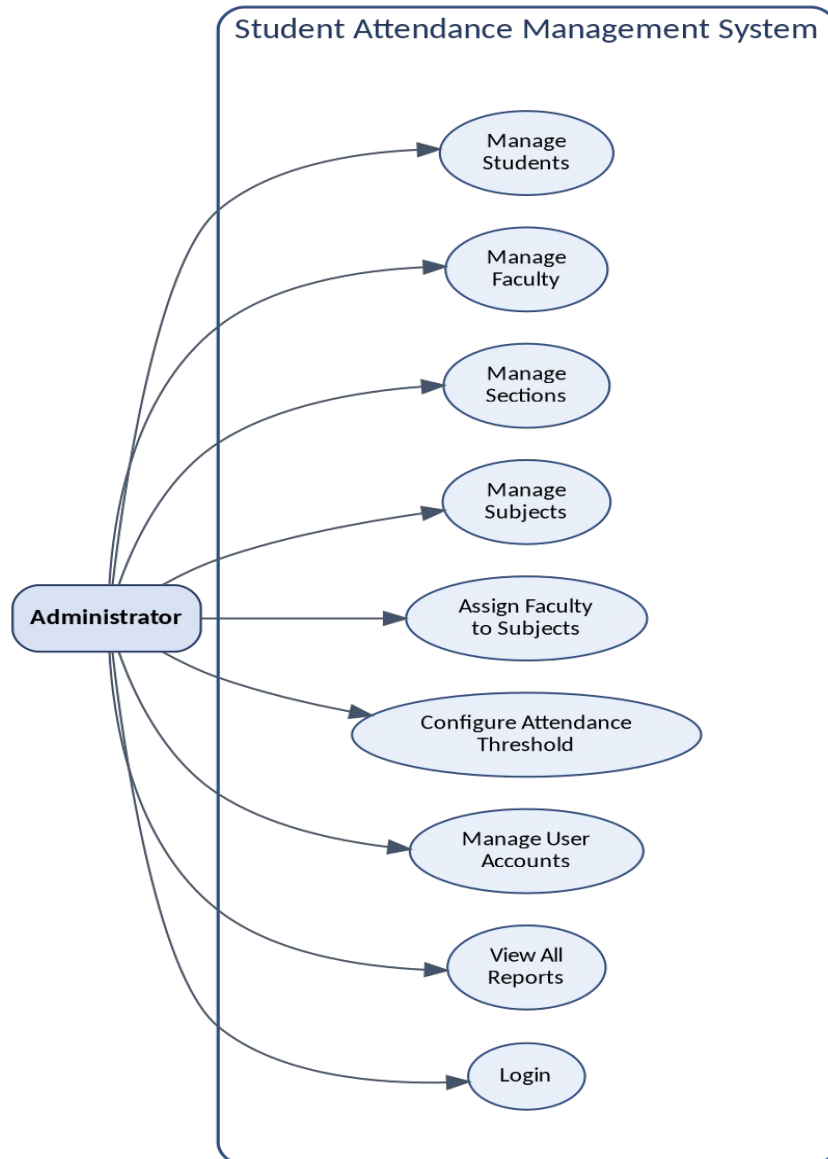


Figure 3.1 — Administrator Use-Case Diagram (reproduced from SRS Figure 7.1)

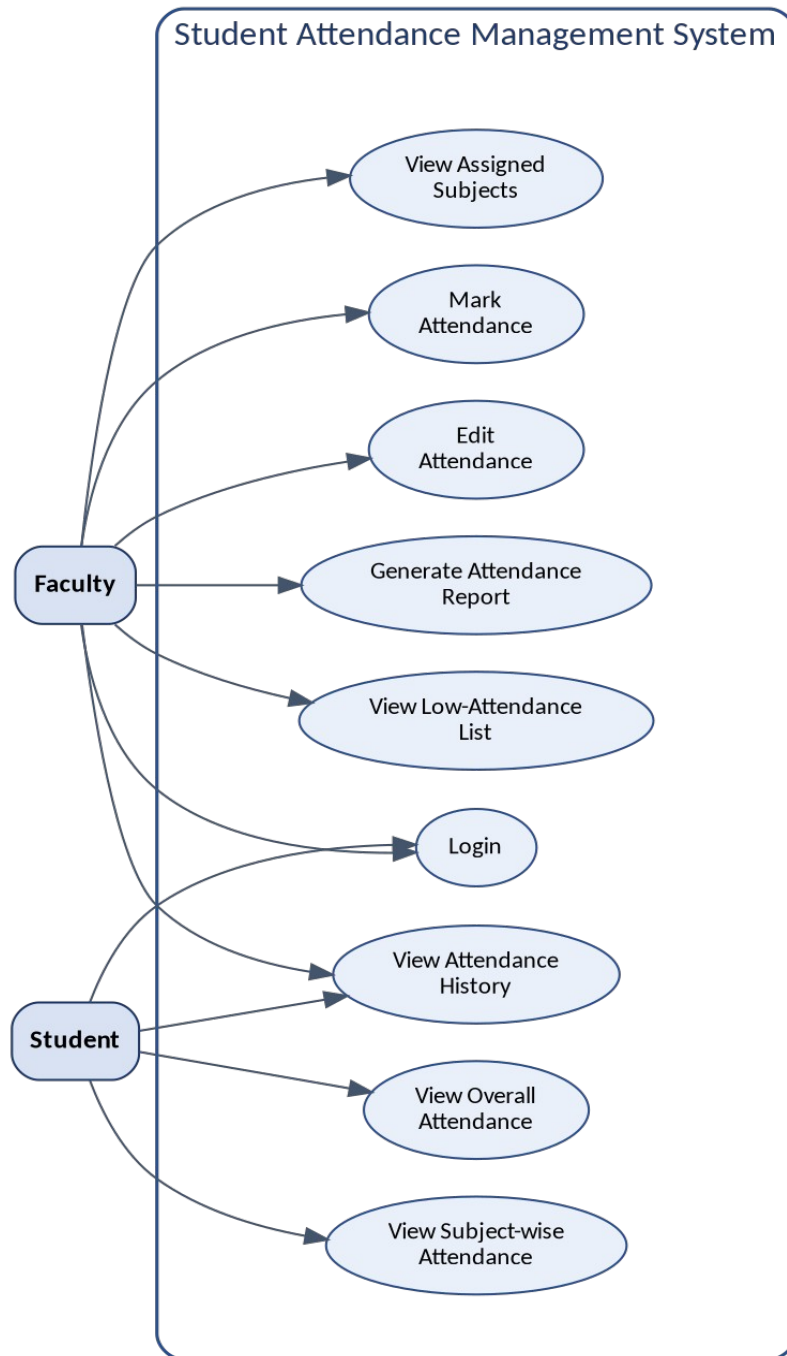


Figure 3.2 — Faculty and Student Use-Case Diagram (reproduced from SRS Figure 7.2)

3.3.2 Component Diagram

The component diagram below shows SAMS' internal module structure as a layered architecture: clients interact through Django Templates (presentation layer), which route to five Django view modules (application layer), each of which reads and writes through the Django ORM to a single SQLite database (data layer).

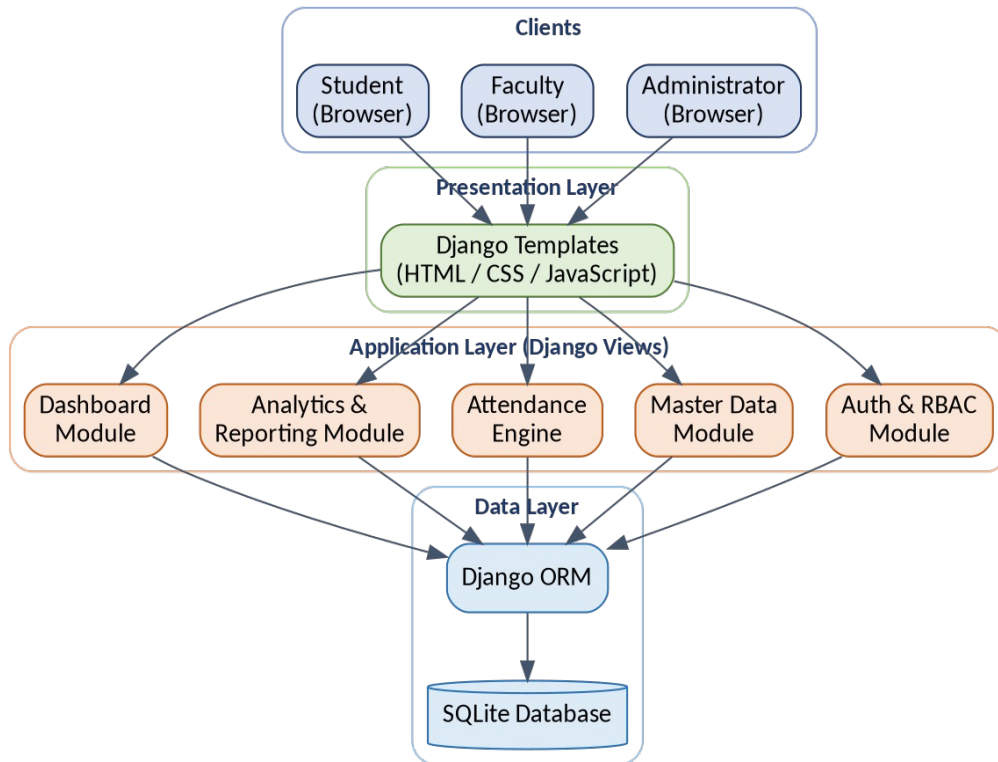


Figure 3.3 — Component Diagram

3.4 Component Descriptions

Component	Description
Django Templates	Server-rendered HTML/CSS/JavaScript presentation layer; one shared base template for consistent navigation across all four developers' pages
Auth & RBAC Module	Handles login/logout, session management, password hashing, CSRF protection, and server-side role enforcement (SAMS-F-001-008, SAMS-SR-001-004)
Master Data Module	Administrator CRUD for Students, Faculty, Sections, and Subjects, and faculty-to-subject-section assignment (SAMS-F-009-021)
Attendance Engine	Faculty-facing roster selection, Present/Absent marking, duplicate prevention, and the authorized edit window (SAMS-F-022-027)
Analytics & Reporting Module	Subject-wise/overall percentage calculation, low-attendance threshold logic, and role-scoped reports (SAMS-F-028-037)
Dashboard Module	Role-specific summary views for Administrator, Faculty, and Student (SAMS-F-038-040)
Django ORM	Single data-access layer through which every module reads/writes; the only path to the database, per SAMS-SR-005
SQLite Database	File-based relational store holding all nine entities defined in SRS Appendix A

3.5 Chosen Architecture Pattern and Rationale

A layered (presentation / application / data) architecture, implemented directly via Django's Model-View-Template (MVT) pattern, was chosen for SAMS.

- Each layer has a single responsibility, which keeps the four-person team's parallel workstreams (Section 6 of the Project Plan) cleanly separated at the application layer, while all modules share the same presentation and data layers.
- A monolithic Django project is appropriate at this scale; a microservices architecture was considered and rejected as unnecessary complexity for a single-institution, classroom-scale academic project with no independent-scaling requirement.
- Django's MVT pattern maps directly onto this layering with no additional framework needed, consistent with the SRS constraint against introducing unnecessary technology.

3.6 Technology Stack & Data Stores

Layer	Technology
Backend / Application	Python 3.x, Django
Presentation	Django Templates, HTML5, CSS, vanilla JavaScript
Data Store	SQLite (single file-based relational database)
Data Access	Django ORM (no raw SQL in application code)
Server	Django development server; optionally WSGI on a local network for demonstration

3.7 Risks & Mitigations

Risk	Mitigation
SQLite's limited concurrent-write support causes contention if multiple faculty mark attendance simultaneously	Acceptable at classroom scale (SRS Section 8); schema is normalized so migration to PostgreSQL would not require a redesign (SAMS-NF-005)
A module depending on another is blocked (e.g. Analytics needs Attendance data)	Seed dummy attendance data early, per the Concurrent Testing approach in the Project Plan
Inconsistent UI across four independently-built modules	A single shared base template and navigation component is built first (WBS 1.0) and reused by all views
Security logic duplicated or bypassed in a hurriedly-built view	All RBAC/CSRF/hashing logic centralised in the Auth & RBAC module; no module queries the database directly outside the ORM

3.8 Traceability to Requirements

A representative sample of requirement-to-component traceability is shown below; the full mapping is maintained in the SRS Requirements Traceability Matrix (Section 8).

Requirement	Component
SAMS-F-001-008, SAMS-SR-001-004 (Authentication & Security)	Auth & RBAC Module
SAMS-F-009-021 (Student/Faculty/Section/Subject Management)	Master Data Module
SAMS-F-022-027 (Attendance Marking)	Attendance Engine
SAMS-F-028-037 (Calculation, Reports, Low-Attendance)	Analytics & Reporting Module
SAMS-F-038-040 (Dashboards)	Dashboard Module

3.9 Security Architecture

Threat modelling for SAMS follows the STRIDE framework, mapped to the security requirements already defined in SRS Section 5.1:

- Spoofing → mitigated by Django's session-based authentication (SAMS-F-001-004); no shared or default credentials.
- Tampering → all database writes go through the Django ORM (SAMS-SR-005), preventing direct/malformed SQL manipulation.
- Repudiation → attendance records capture marked_at/modified_at timestamps and the marking faculty_id (SAMS-F-024, SRS Appendix A), giving an audit trail for who changed what and when.
- Information Disclosure → Student report views are scoped to the logged-in student's own student_id (SAMS-F-033); CSRF tokens protect all state-changing forms (SAMS-SR-004).
- Denial of Service → out of scope at this academic scale (SRS Section 8 constraints); not a design goal for this project.
- Elevation of Privilege → every protected view enforces role checks server-side in Django views, not only hidden in the frontend (SAMS-SR-003).

4. Design

4.1 Design Overview

SAMS is designed as a set of loosely-coupled Django apps, one per functional area (auth, master data, attendance, analytics, dashboard), each with its own models, views, and templates extending a shared base template. This keeps each of the four team members' work independently testable while integrating through a common data layer.

4.2 UML Sequence Diagrams

Two sequence diagrams are provided, covering the system's two most central flows: a Faculty marking attendance, and a Student viewing their attendance report.

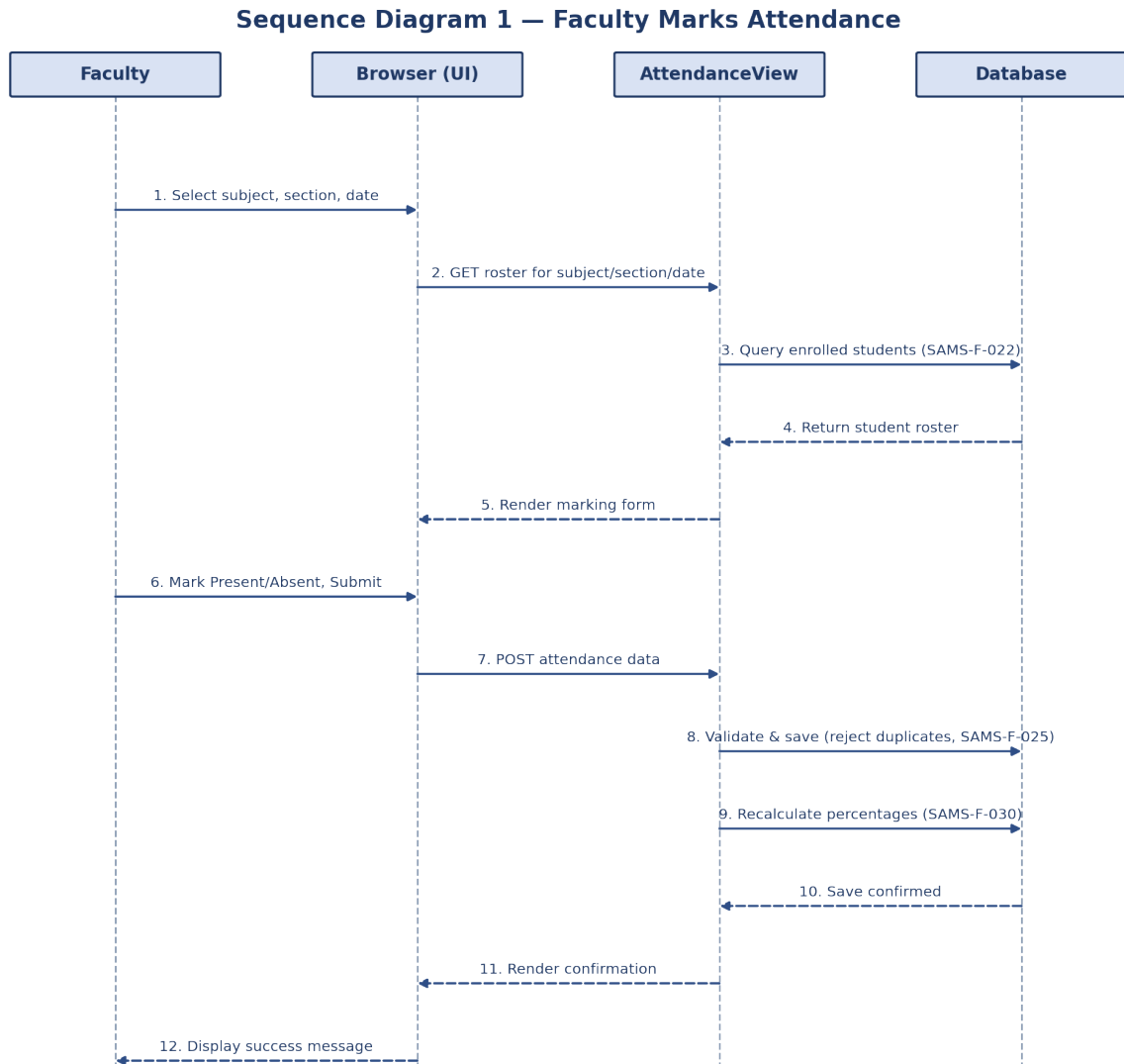


Figure 4.1 — Sequence Diagram: Faculty Marks Attendance

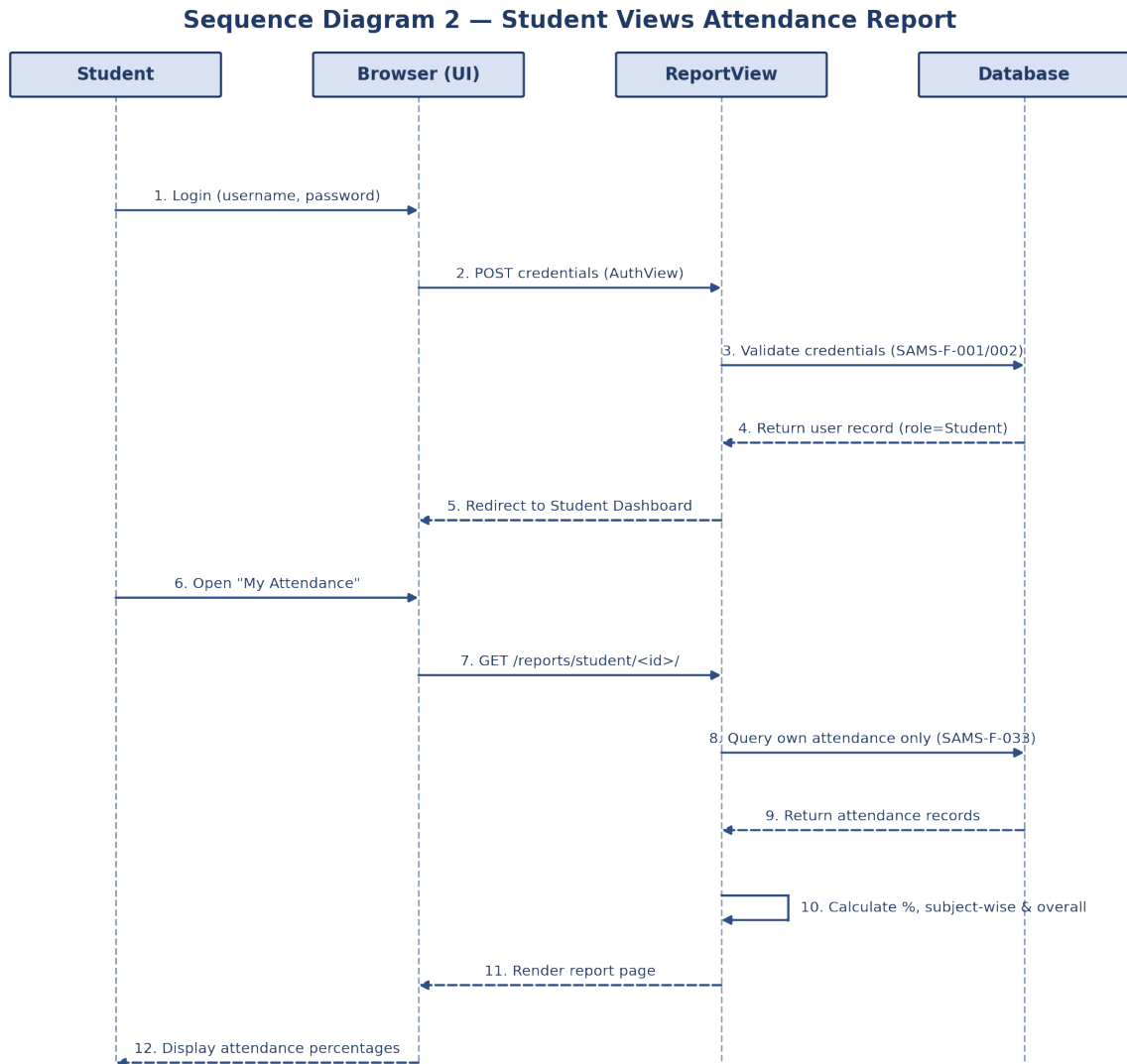


Figure 4.2 — Sequence Diagram: Student Views Attendance Report

4.3 API Design

SAMS is a server-rendered Django Templates application rather than a REST API, so "interfaces" here are Django view endpoints (URL routes) rather than a JSON API. The two most central endpoints are defined below, covering the Attendance Engine and Analytics & Reporting components.

Endpoint: `/attendance/mark/`

- Method: GET (load roster) / POST (submit marking)
- Request (POST): { subject_id, section_id, date, attendance: [{student_id, status}] }
- Response: Rendered confirmation page, or re-rendered form with field errors
- Errors: 403 if Faculty is not assigned to the subject/section (SAMS-F-007); 400 if date is outside the current term (SAMS-F-027)

Endpoint: `/reports/student/<student_id>/`

- Method: GET
- Request: student_id (path parameter), optional date-range query parameters
- Response: Rendered report page with subject-wise and overall attendance percentages

- Errors: 403 if the logged-in user is a Student requesting a student_id other than their own (SAMS-F-033)

4.4 Error Handling, Logging & Monitoring

Form validation errors are displayed inline, in plain language, next to the relevant field (SAMS-NF-006). No sensitive information (passwords, session tokens) is written to application logs. For this academic-scale deployment, monitoring is limited to manual review of the Django development server console during testing; production-grade monitoring is out of scope (see Open Issues, Section 4.6).

4.5 UX Design

A single shared base template defines consistent navigation, labelling, and layout across all pages for a given role (SAMS-NF-003). Attendance marking is designed to be completed in a small number of clicks: select subject/section/date, then mark all students on one page. Low-attendance percentages are visually highlighted on screen (SRS FR-LOW-05 / SAMS-F-036 context) so Faculty and Administrators can spot at-risk students at a glance.

4.6 Open Issues & Next Steps

- Production-grade monitoring/alerting is not designed in this version; out of scope for the academic deployment target.
- Future enhancements identified in SRS Appendix C (email/SMS notifications, PDF/Excel export, PostgreSQL migration, QR-code attendance) remain unscheduled beyond this project's semester timeline.

5. Appendices

5.1 Glossary

SAMS, SRS, SAD, UML, ORM, RBAC, CSRF, STRIDE, WBS, MVT. Full definitions are provided in the Phase-1 SRS, Section 1.4.

5.2 References

- Software Requirements Specification (SRS) — Student Attendance Management System, Version 1.0: [GitHub — SRS-Student Attendance Management System.pdf](#)
- Jackfruit Phase-2 — Project Plan Document, Version 1.1
- IEEE 42010 — Systems and software engineering — Architecture description (used as a general structural reference, adapted for a college mini-project)

5.3 Tools

- draw.io, Graphviz — UML use-case and component diagrams
- Python / Matplotlib — UML sequence diagrams
- GitHub — version control and document hosting (SRS linked in Section 2.2)